TRAVAIL PRATIQUE 1

CampusRate

Conception d’une API REST professionnelle

420-514

Collecte et interprétation des données

Techniques de l’informatique

Automne 2026

Informations générales

Élément	Information
Travail	TP1 : CampusRate
Réalisation	**Individuelle**
Durée	**Deux semaines (voir la date de remise sur Léa)**
Remise	Sur la boite de remise (sur Léa)
Technologies	NestJS, TypeScript, JSON, Swagger/OpenAPI, Postman

1. Mandat

On doit concevoir **CampusRate**, une API REST qui permet à la communauté étudiante de consulter des endroits ou services du campus et de publier des appréciations accompagnées d'une note.

Le TP est réalisé **individuellement**. Chaque personne doit concevoir son propre contrat d'API, produire son code et être en mesure de justifier ses décisions.

Le travail évalue principalement la capacité à concevoir un contrat d'API professionnel. **Les noms techniques des ressources, les URI, les méthodes HTTP et les codes de réponse.** On doit les choisir adéquatement, les appliquer de façon uniforme et les documenter.

La solution doit également :

1. être organisée par fonctionnalité dans NestJS;
2. valider les entrées et appliquer les règles métier;
3. persister les données dans un fichier JSON local;
4. gérer les erreurs avec un format uniforme;
5. produire une documentation Swagger/OpenAPI complète;
6. être vérifiée manuellement avec Swagger UI ou Postman;
7. pouvoir être installée à partir d'un README professionnel.

2. Données imposées

CampusRate manipule deux concepts : un endroit évalué et une appréciation associée à cet endroit. On doit choisir leurs noms de ressources en anglais et les utiliser avec cohérence dans les URI, le code et OpenAPI.

2.1 Endroit évalué

Attribut	Type et exemple	Création	Règle
`id`	chaîne, `plc_01JABC123`	Interdit	Généré par le serveur, non modifiable
`name`	chaîne, `Bibliothèque principale`	Requis	Texte significatif
`description`	chaîne, `Espace calme avec prises.`	Requis	Longueur raisonnable
`category`	chaîne, `STUDY_SPACE`	Requis	Valeur autorisée
`address`	chaîne, `Pavillon A, local A-210`	Requis	Emplacement compréhensible
`services`	tableau, `["WIFI", "POWER_OUTLETS"]`	Facultatif	Tableau vide par défaut, sans doublons
`status`	chaîne, `ACTIVE`	Facultatif	Valeur par défaut documentée
`averageRating`	nombre ou `null`, `4.25`	Interdit	Calculé par le serveur
`reviewCount`	entier, `12`	Interdit	Calculé par le serveur
`createdAt`, `updatedAt`	date ISO 8601	Interdit	Produites par le serveur

Catégories minimales : `STUDY_SPACE`, `LIBRARY`, `FOOD_SERVICE`, `SPORTS`, `STUDENT_SERVICE`, `COMPUTER_LAB`, `OTHER`.

États minimaux : `ACTIVE`, `TEMPORARILY_CLOSED`, `INACTIVE`.

2.2 Appréciation

Attribut	Type et exemple	Création	Règle
`id`	chaîne, `rev_01JXYZ789`	Interdit	Généré par le serveur, non modifiable
`placeId`	chaîne, `plc_01JABC123`	Selon le contrat conçu	Référence un endroit existant, non modifiable
`authorName`	chaîne, `Samira`	Requis	Nom ou pseudonyme
`rating`	entier, `4`	Requis	Valeur de 1 à 5
`comment`	chaîne, `Calme et Wi-Fi stable.`	Requis	Longueurs minimale et maximale documentées
`createdAt`, `updatedAt`	date ISO 8601	Interdit	Produites par le serveur

3. Capacités obligatoires

On doit concevoir le contrat HTTP qui permet de :

1. créer, lister et consulter les endroits;
2. modifier partiellement et supprimer un endroit lorsque les règles le permettent;
3. publier et lister les appréciations associées à un endroit;
4. consulter, modifier partiellement et supprimer une appréciation;
5. filtrer et paginer la collection des endroits.

Pour chaque capacité, on doit déterminer et documenter : la méthode HTTP, l'URI, les paramètres, le corps accepté, le code de succès, la représentation retournée et les erreurs possibles.

4. Exigences de design d'API

Le contrat doit respecter les exigences suivantes :

1. ressources nommées en anglais, au pluriel, en minuscules et sans verbe d'action;
2. version majeure incluse dans le chemin selon une convention uniforme;
3. méthodes HTTP choisies selon leur sémantique;
4. `id` et `placeId` utilisés comme noms des paramètres de chemin lorsqu'ils sont nécessaires;
5. imbrication limitée à l'expression d'une relation réelle entre les concepts;
6. décision justifiée entre URI imbriquée, URI indépendante ou combinaison des deux;
7. en-tête `Location` après une création réussie;
8. corps vide lorsque le code de succès choisi n'autorise aucun contenu;
9. URI stables et indépendantes de l'organisation interne du code;
10. format JSON cohérent et dates en ISO 8601;
11. codes de succès et d'erreur adaptés à chaque situation;
12. collection vide représentée par un tableau vide et non par une erreur.

Le README doit contenir un court tableau justifiant les noms de ressources, le versionnement, l'imbrication et les principaux choix de codes HTTP.

5. Filtrage et pagination

L'opération de collection doit accepter les paramètres imposés suivants :

Paramètre	Type	Exemple	Fonction
`category`	chaîne contrôlée	`STUDY_SPACE`	Filtre exact
`page`	entier	`1`	Page demandée
`limit`	entier	`10`	Taille de page

Le filtre et la pagination doivent pouvoir être combinés. `page` et `limit` doivent être validés et une limite maximale doit être imposée.

La réponse doit séparer les données des métadonnées de pagination :

```
{
  "data": [],
  "pagination": {
    "page": 1,
    "limit": 10,
    "totalItems": 0,
    "totalPages": 0
  }
}
```

6. Validation et règles métier

On doit utiliser des DTO adaptés à la création, à la modification et aux paramètres de requête. La validation globale doit refuser les types, formats, valeurs et propriétés inconnues non autorisés.

Règles obligatoires :

1. une appréciation désigne toujours un endroit existant;
2. `id`, `placeId`, les dates et les valeurs calculées ne peuvent pas être imposés par le client;
3. la note moyenne et le nombre d'appréciations reflètent les données existantes;
4. un endroit sans appréciation expose `averageRating: null` et `reviewCount: 0`;
5. toute modification ou suppression d'une appréciation met à jour les valeurs calculées;
6. la suppression d'un endroit qui possède des appréciations est refusée comme un conflit avec son état actuel;
7. les catégories et états appartiennent aux listes autorisées.

7. Gestion uniforme des erreurs

Les erreurs doivent utiliser le type de contenu `application/problem+json` et le format Problem Details : `type`, `title`, `status`, `detail` et `instance`.

On doit choisir le code HTTP approprié pour une entrée invalide, une ressource inexistante, un conflit avec l'état courant et une erreur technique inattendue. Une erreur interne ne doit exposer ni pile d'exécution, ni chemin local, ni contenu brut du fichier.

Un filtre global doit transformer les exceptions attendues et inattendues selon ce format.

8. Persistance JSON

Les données doivent demeurer disponibles après un redémarrage. La lecture et l'écriture doivent :

- utiliser les API asynchrones de `node:fs/promises`;
- être isolées dans un service ou dépôt de données;
- initialiser proprement le fichier lorsqu'il est absent;
- détecter un JSON invalide et produire une erreur contrôlée;
- utiliser un chemin configurable, par exemple `DATA_FILE_PATH`.

Le format du fichier et la séparation entre la logique métier et l'accès aux données doivent être brièvement expliqués.

9. Documentation et vérification manuelle

Swagger/OpenAPI doit décrire fidèlement les ressources, opérations, paramètres, corps, champs requis, exemples, réponses, erreurs, pagination et ProblemDetails. Une personne doit pouvoir utiliser l'API sans consulter le code source.

Une collection Postman exportée ou un document équivalent doit présenter de **8 à 10 scénarios représentatifs**. On doit notamment couvrir la création et la consultation des deux concepts, une modification, une suppression permise, une note invalide, une ressource inexistante, un conflit de suppression, le filtre, la pagination, une erreur Problem Details et la persistance après redémarrage. Certains scénarios peuvent vérifier plusieurs exigences à la fois.

10. Organisation et configuration

On doit séparer les contrôleurs, les services métier, les DTO, la couche de persistance et le filtre global regroupé dans `common`.

Un fichier `.env.example` doit documenter au moins `PORT` et `DATA_FILE_PATH`. Le fichier `.env`, les secrets, `node_modules` et les artefacts inutiles ne doivent pas être versionnés. L'application doit refuser une configuration obligatoire absente ou invalide.

11. README, Git et remise

Le README présente CampusRate comme un vrai projet public : objectif, fonctionnalités, technologies, installation, configuration, démarrage, lint, compilation, Swagger UI, contrat général et limites connues.

Le développement doit utiliser une issue parente, des sous-issues réalisables, une branche `feature` associée à l'issue parente et, au besoin, des branches `task`. Les commits et la pull request doivent être descriptifs et référencer les bonnes issues.

La remise comprend :

1. le dépôt GitHub et le code source;
2. package-lock.json et .env.example;
3. le README professionnel;
4. la spécification OpenAPI ou sa configuration de génération;
5. la collection Postman ou le document d'essais manuels;
6. la pull request finale prête à être évaluée.
7. Le document de la déclaration de l'intégrité intellectuelle (voir la section suivante pour plus de détail).

Avant la remise, on exécute :

```
npm ci
npm run lint
npm run build
```

12. Utilisation encadrée de l'intelligence artificielle

L'intelligence artificielle générative peut servir d'aide à la rédaction, à la compréhension, à la révision et au diagnostic. On peut notamment l'utiliser pour structurer une première version du README, reformuler une documentation, expliquer un message d'erreur, commenter une solution déjà produite ou comparer des choix envisagés.

Les décisions évaluées concernant le design de l'API demeurent toutefois sous la responsabilité de l'étudiant. On doit concevoir, valider, justifier et être en mesure d'expliquer :

1. le nom des ressources et la structure des URI;
2. le versionnement, les méthodes HTTP et les codes de statut;
3. les paramètres, les relations et les représentations JSON;
4. les DTO, les validations et le format des erreurs;
5. le code conservé dans le projet.

Il est interdit de remettre une API entièrement produite par une IA ou tout code sans en maîtriser son fonctionnement. On ne doit transmettre à aucun outil externe un secret, un jeton, un fichier `.env`, un renseignement personnel ou une donnée confidentielle.

Déclaration avec IAGraphie

Chaque étudiant doit remplir la déclaration d'intégrité demandée. Lorsqu'une IA est utilisée, on doit également produire la déclaration avec le générateur IAGraphie du Cégep et y inclure les liens vers les conversations utilisées ou y partager les requêtes effectuées, conformément aux consignes de l'outil (voir la vidéo explicative : 2 min Youtube_v2.mp4).

La déclaration de l'intégrité intellectuelle doit être effectuée à l'Aide de l'outil suivant : **IAGraphie**. Le résultat de cette déclaration doit être joint séparément à la remise; il ne doit pas être publié dans le README du dépôt public.

Lorsqu'une IA est utilisée, on remet également :

1. un prompt représentatif;
2. une courte explication de l'aide obtenue;
3. un commentaire de trois à cinq lignes présentant au moins deux vérifications, corrections ou adaptations effectuées.

L'étudiant demeure responsable de l'exactitude de tout le contenu remis. Une documentation générée qui décrit des commandes, des routes ou des fonctionnalités inexistantes sera considérée comme incorrecte. Une courte validation individuelle peut être réalisée afin de vérifier la compréhension du projet et des choix de conception.

13. Grille d'évaluation

Critère	Points
Design REST, versionnement et contrat HTTP	25
Gestion des deux concepts et de leur relation	20
Filtrage et pagination	7
Validation et règles métier	13
Persistance JSON et séparation des responsabilités	10
Gestion uniforme des erreurs	5
Swagger/OpenAPI, README, essais manuels et usage responsable de l'IA	15
Qualité du code, configuration et Git	5
Total	**100**

Définition de terminé (liste des vérifications suggérée)

1. [ ] Toutes les capacités obligatoires sont fonctionnelles.
2. [ ] Les choix de design sont cohérents, justifiés et documentés.
3. [ ] Les entrées invalides et les erreurs sont traitées uniformément.
4. [ ] Le fichier JSON conserve les données après un redémarrage.
5. [ ] Swagger correspond au comportement réel.
6. [ ] Les scénarios manuels sont reproductibles.
7. [ ] Le lint et la compilation réussissent.
8. [ ] Le README et la pull request sont complets.
9. [ ] L'utilisation de l'IA est déclarée avec IAGraphie.